

# EcoVision

AI-Based Waste Classification System

Computer Vision Project Report

**Domain:** Computer Vision / Artificial Intelligence

**Technology:** Python, TensorFlow/Keras, MobileNetV2, Streamlit

Prepared for VITyarthi – Build Your Own Project

# 1. Introduction

EcoVision is a computer vision application designed to classify common waste items from images. The system uses transfer learning with MobileNetV2 and provides a simple Streamlit interface where a user can upload an image and receive a predicted waste category, confidence score, and class probability distribution.

The project combines machine learning, image preprocessing, modular Python development, evaluation, testing, documentation, and Git/GitHub version control.

# 2. Problem Statement

Improper waste segregation can make recycling and waste management more difficult. Manual identification of waste categories can also be time-consuming and inconsistent. EcoVision addresses this problem by automatically classifying an input waste image into one of six categories: cardboard, glass, metal, paper, plastic, or trash.

# 3. Objectives

- Build an image-based waste classification system.
- Apply transfer learning using MobileNetV2.
- Provide a user-friendly interface for image upload and prediction.
- Display confidence and class probabilities for each prediction.
- Evaluate the trained model using accuracy, precision, recall, F1-score, and a confusion matrix.
- Implement the project as modular, maintainable Python files.
- Provide testing and documentation suitable for an academic software project.

# 4. Scope and Target Users

The project focuses on image-based classification of common household waste represented by six classes. Supported input formats are JPG, JPEG, and PNG.

**Target users:** students learning computer vision, educational institutions, environmental awareness projects, and users interested in waste segregation.

# 5. Functional Requirements

| ID  | Requirement                                                 |
|-----|-------------------------------------------------------------|
| FR1 | Upload a waste image.                                       |
| FR2 | Convert, resize, and normalize the image before prediction. |
| FR3 | Classify the image into one of six waste categories.        |
| FR4 | Display the prediction confidence.                          |
| FR5 | Display probabilities for all six categories.               |
| FR6 | Maintain prediction history during the application session. |
| FR7 | Display basic prediction analytics.                         |
| FR8 | Provide model evaluation and testing modules.               |

# 6. Non-Functional Requirements

- **Performance:** use a lightweight MobileNetV2 architecture for practical inference.
- **Usability:** provide a simple Streamlit upload interface.

- **Reliability:** validate model loading, preprocessing, and prediction through tests.
- **Maintainability:** separate configuration, preprocessing, utilities, training, evaluation, and application logic.
- **Resource efficiency:** use transfer learning instead of training a large CNN from scratch.
- **Error handling:** handle missing or invalid image inputs in the command-line prediction module.
- **Scalability:** allow additional classes and model improvements through modular design.
- **Portability:** define dependencies in requirements.txt and use Python-based components.

## 7. System Architecture

The system follows a modular machine-learning workflow:

User Image → Streamlit Application → Image Preprocessing → MobileNetV2 Model → Classification Probabilities → Predicted Class + Confidence → History/Analytics

### Training workflow

TrashNet dataset → 80/20 train-validation split → image resizing and normalization → MobileNetV2 feature extraction → GlobalAveragePooling2D → Dense(128, ReLU) → Dense(6, Softmax) → model training → waste\_classifier.keras

### Main modules

- train.py – trains the classification model.
- predict.py – performs command-line prediction.
- app.py – provides the Streamlit user interface.
- config.py – stores shared configuration and class definitions.
- preprocess.py – performs image conversion, resizing, normalization, and batching.
- utils.py – loads the model and performs predictions.
- evaluate.py – generates evaluation metrics and a confusion matrix.
- test\_model.py – checks model loading, preprocessing, output classes, and prediction.

## 8. Process Flow

1. User uploads a JPG, JPEG, or PNG image.
2. The image is converted to RGB.
3. The image is resized to 224 × 224 pixels.
4. Pixel values are normalized to the range 0–1.
5. The processed image is passed to the trained MobileNetV2 model.
6. Softmax probabilities are generated for six classes.
7. The class with the highest probability is selected.
8. The application displays the class, confidence, probabilities, and session history.

## 9. Dataset

The project uses the TrashNet dataset with six categories: cardboard, glass, metal, paper, plastic, and trash. The training pipeline uses an 80/20 validation split.

| Subset | Images |
|--------|--------|
|--------|--------|

|            |      |
|------------|------|
| Training   | 2024 |
| Validation | 503  |
| Total      | 2527 |

## 10. Model Selection and Methodology

MobileNetV2 pretrained on ImageNet was selected for transfer learning. Its convolutional base is frozen, and a custom classification head is added. This reduces the amount of training required while providing useful pretrained visual features.

Input images are resized to  $224 \times 224$  and normalized by dividing pixel values by 255. The model uses GlobalAveragePooling2D, a Dense layer with 128 ReLU neurons, and a six-class softmax output layer. Training uses the Adam optimizer with learning rate 0.001, categorical cross-entropy loss, batch size 16, and 5 epochs.

## 11. Results

| Metric                  | Result |
|-------------------------|--------|
| Final training accuracy | 97.73% |
| Validation accuracy     | 77.73% |
| Validation images       | 503    |
| Macro precision         | 0.75   |
| Macro recall            | 0.74   |
| Macro F1-score          | 0.75   |
| Weighted precision      | 0.78   |
| Weighted recall         | 0.78   |
| Weighted F1-score       | 0.78   |

### Class-wise evaluation

| Class     | Precision | Recall | F1   |
|-----------|-----------|--------|------|
| Cardboard | 0.89      | 0.74   | 0.81 |
| Glass     | 0.72      | 0.81   | 0.76 |
| Metal     | 0.74      | 0.85   | 0.79 |
| Paper     | 0.85      | 0.87   | 0.86 |
| Plastic   | 0.77      | 0.67   | 0.72 |
| Trash     | 0.56      | 0.52   | 0.54 |

The results show a generalization gap between training accuracy (97.73%) and validation accuracy (77.73%). This indicates some overfitting. Paper achieved an F1-score of 0.86 in this evaluation, while trash had the lowest F1-score at 0.54.

## 12. Confusion Matrix

The evaluation produced the following confusion matrix. Rows represent actual classes and columns represent predicted classes in the order: cardboard, glass, metal, paper, plastic, trash.

| Actual \ Predicted | Card | Glass | Metal | Paper | Plastic | Trash |
|--------------------|------|-------|-------|-------|---------|-------|
| 59                 | 1    | 4     | 13    | 1     | 2       |       |
| 0                  | 81   | 12    | 2     | 5     | 0       |       |

|   |    |    |     |    |    |  |
|---|----|----|-----|----|----|--|
| 0 | 7  | 70 | 1   | 3  | 1  |  |
| 6 | 1  | 2  | 103 | 2  | 4  |  |
| 1 | 21 | 5  | 1   | 64 | 4  |  |
| 0 | 2  | 2  | 1   | 8  | 14 |  |

## 13. Testing

The `test_model.py` module validates core model functionality. Tests cover model file existence, model loading, output class count, image preprocessing shape, and prediction output.

The implemented tests successfully verified image preprocessing and prediction. A sample prediction test returned a valid class and confidence value, demonstrating that the end-to-end inference path is operational.

## 14. Implementation and GitHub

The project is implemented as eight meaningful Python modules plus documentation. The dataset is excluded from version control using `.gitignore`, while the trained model is included because its size is approximately 11.1 MB and therefore fits within GitHub's individual file limit.

Repository: <https://github.com/abhaycosmicstorm/EcoVision>

## 15. Challenges

- Selecting a practical transfer-learning architecture for limited project resources.
- Maintaining consistency between training and evaluation preprocessing.
- Handling the difference between training and validation performance.
- Organizing the application into multiple maintainable modules.
- Integrating the trained model into a Streamlit interface.
- Configuring Git/GitHub and keeping the dataset out of version control.

## 16. Learnings

- Transfer learning can be used to build practical image classifiers efficiently.
- Consistent preprocessing is important during both training and evaluation.
- Accuracy alone is not sufficient; precision, recall, F1-score, and confusion matrices provide additional insight.
- Modular project structure improves maintainability and testing.
- Git and GitHub are important for version control and project documentation.
- A machine-learning project benefits from separating training, inference, evaluation, testing, and UI logic.

## 17. Future Enhancements

- Increase dataset diversity and size.
- Apply stronger data augmentation.
- Fine-tune selected MobileNetV2 layers.
- Add real-time camera-based classification.
- Support multiple waste objects using object detection.
- Add waste segregation and recycling recommendations.

- Deploy the application to a cloud platform.
- Develop a mobile version.

## 18. Conclusion

EcoVision demonstrates a complete computer vision workflow from dataset preparation and transfer-learning-based model training to evaluation, testing, modular implementation, and deployment through a Streamlit interface. The system successfully classifies images into six waste categories and provides confidence and probability information. The project also incorporates software engineering practices including modular architecture, requirements documentation, UML/architecture documentation, automated testing, Git version control, and GitHub documentation.

## 19. References

- TrashNet dataset – Gary Thung and Mindy Yang.
- TensorFlow/Keras documentation – image classification and transfer learning.
- MobileNetV2 – Sandler et al., MobileNetV2: Inverted Residuals and Linear Bottlenecks.
- Scikit-learn documentation – classification metrics and confusion matrix.
- Streamlit documentation – application development and UI components.